LiteLLM Deployment on wavestone-ai-librechat

Status: ☑ Deployed and Running (Stateless Mode) **Date**: November 10, 2025 **Project**: wavestone-ai-librechat **Region**: europe-west1

Deployment Summary

LiteLLM has been successfully deployed to the GKE cluster in stateless mode.

Access Information

Resource	Value
External IP	`35.205.93.210`
Admin UI	http://35.205.93.210/ui
API Endpoint	http://35.205.93.210
Swagger Docs	http://35.205.93.210/docs
Master Key	`sk-0m+gLRphHEY8d/Dl38K2A+d1bP5u+OCXJOHnR2o/MvLs4B/EOC+dgGwS2hEJmbQd`

Architecture

```
Internet
    ↓
GCP LoadBalancer (35.205.93.210:80)
    ↓
LiteLLM Service (ClusterIP 34.118.232.196:4000)
    ↓
LiteLLM Pod (1 replica, stateless)
```

Configuration

Current Setup

- **Mode**: Stateless (no database persistence)
- **Namespace**: `litellm`
- **Cluster**: `librechat-cluster` (europe-west1-b)
- **Image**: `europe-west1-docker.pkg.dev/wavestone-ai-librechat/litellm-repo/litellm:latest`
- **Resources**:
  - CPU Request: 250m / Limit: 1000m
  - Memory Request: 512Mi / Limit: 2Gi
- **Workers**: 1

- **Log Level**: INFO

Important Limitations (Stateless Mode)

⚠ **No Persistence**: Current deployment runs without a database:

- ❌ API keys and configurations are lost on pod restart
- ❌ No user management or authentication tracking
- ❌ No usage statistics or billing tracking
- ❌ Admin UI has limited functionality
- ☑ Proxy functionality works for configured models

Using LiteLLM

Admin Interface

Access the admin UI at: **http://35.205.93.210/ui**

Use the master key for authentication.

API Proxy Usage

Test Connection

```
curl http://35.205.93.210/ \
  -H "Authorization: Bearer sk-
0m+gLRphHEY8d/Dl38K2A+d1bP5u+OCXJOHnR2o/MvLs4B/EOC+dgGwS2hEJmbQd"
```

Configure Models

Models must be added through the admin UI or by updating the config.yaml ConfigMap.

Example models can be added to `config.yaml`:

```
model_list:
  - model_name: gpt-4
    litellm_params:
      model: openai/gpt-4
      api_key: ${OPENAI_API_KEY}

  - model_name: claude-3-sonnet
    litellm_params:
      model: anthropic/claude-3-sonnet-20240229
      api_key: ${ANTHROPIC_API_KEY}
```

Then update the ConfigMap and restart:

```
kubectl create configmap litellm-config \
  --from-file=config.yaml=config.yaml \
  --namespace=litellm \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl rollout restart deployment/litellm -n litellm
```

Operations

Common Commands

```
# Check pod status
kubectl get pods -n litellm

# View logs
kubectl logs -f deployment/litellm -n litellm

# Restart deployment
kubectl rollout restart deployment/litellm -n litellm

# Check service and external IP
kubectl get svc -n litellm

# Access admin UI
# Open browser to: http://35.205.93.210/ui
```

Updating Configuration

```
# Update config
kubectl edit configmap litellm-config -n litellm

# Restart to apply changes
kubectl rollout restart deployment/litellm -n litellm

# Monitor restart
kubectl rollout status deployment/litellm -n litellm
```

Scaling

```
# Note: Stateless mode can scale horizontally
kubectl scale deployment litellm -n litellm --replicas=2
```

Future Improvements

Adding Database Persistence

To enable full functionality with persistence, you can:

1. **Option A: Use the deployed PostgreSQL**
   - PostgreSQL StatefulSet is already deployed (litellm-postgres-0)
   - Uncomment database configuration in config.yaml
   - Add DATABASE_URL environment variable
   - Requires fixing Prisma migration issues
2. **Option B: Use Cloud SQL**
   - Create a Cloud SQL PostgreSQL instance
   - Configure private IP connection from GKE
   - More reliable for production use
3. **Option C: Use SQLite with PVC**
   - Simple but not recommended for production
   - Works only with single replica

Recommended Next Steps

1. Set up proper database persistence
2. Configure authentication and user management
3. Add model configurations for your use case
4. Set up monitoring and alerting
5. Configure HTTPS with proper domain name
6. Integrate with LibreChat if needed

Infrastructure Details

Deployed Resources

Resource	Name	Type	Status
Namespace	litellm	Namespace	☑ Active
Deployment	litellm	Deployment	☑ Running (1/1)
Service	litellm	ClusterIP	☑ Active
Service	litellm-external	LoadBalancer	☑ Active (35.205.93.210)
ConfigMap	litellm-config	ConfigMap	☑ Active
Secret	litellm-secrets	Secret	☑ Active
StatefulSet	litellm-postgres	PostgreSQL	☑ Running (unused)

Resource	Name	Type	Status
PVC	litellm-data-pvc	5Gi	☑ Bound (unused)

Cost Estimation

Monthly Cost (Stateless Mode):

- LoadBalancer: ~$18/month
- Compute (250m CPU, 512Mi RAM): ~$5/month
- Egress traffic: Variable
- **Total**: ~$25-30/month

Note: PostgreSQL StatefulSet adds ~$8-10/month if kept running.

Troubleshooting

Pod Not Starting

```
# Check pod status
kubectl describe pod -l app=litellm -n litellm

# Check logs
kubectl logs -l app=litellm -n litellm --tail=100
```

Cannot Access UI

1. Verify LoadBalancer has external IP:

```
kubectl get svc litellm-external -n litellm
```

2. Test connectivity:

```
curl http://35.205.93.210/
```

3. Check firewall rules in GCP Console

Configuration Not Applied

```
# Verify ConfigMap
kubectl get configmap litellm-config -n litellm -o yaml

# Force restart
kubectl rollout restart deployment/litellm -n litellm
```

```
# Watch restart
kubectl get pods -n litellm -w
```

Security Notes

- Master key is stored in Kubernetes Secret
- LoadBalancer is publicly accessible (no authentication layer)
- Consider adding Cloud Armor or IAP for production
- Rotate master key regularly
- Use network policies to restrict access if needed

Documentation

- **LiteLLM Official Docs**: https://docs.litellm.ai/
- **Kubernetes Manifests**: `genai-litellm/k8s/`
- **Original Deployment Docs**: See `README.md` for ai-garage-475806 deployment

---

Contact

Deployed by: Claude Code **Date**: November 10, 2025 **Maintainer**: clement.peponnet@wavestone.com

---

Version: 1.0 (Stateless) **Last Updated**: 2025-11-10